

Abeto Messenger

技术调研结论

游戏定位、技术栈、运行时、渲染管线与性能优化方法

调研日期：2026-06-18

输出形式：可下载 DOCX / PDF 报告

核心判断

Messenger 是一款浏览器内运行的轻量 3D 配送/漫游小游戏。其技术路线不是 Unity/Godot WebGL build，而更接近「浏览器 JavaScript runtime + Three.js/WebGL + 自研游戏系统 + 局部 WebAssembly」的组合。性能优化的关键不是单点黑科技，而是从游戏设计、美术生产、资产格式、LOD、内存生命周期和多人规模上同时收紧预算。

1. Executive Summary

Abeto 的 Messenger 是一款免费、浏览器内运行的 3D 配送/漫游小游戏：玩家扮演小邮差，在一个微型球形星球上接配送任务、探索城市/山地/海边等区域，并与其他在线玩家用 3D emoji / stamps 交流。公开资料提到它有 5 个 delivery jobs，支持 PC 和手机浏览器。

技术上，它更像 Abeto 为未来 Web 多人空间做的一次 lightweight realtime 3D 实验：核心栈为 Three.js + WebGL + vanilla JavaScript；部分性能敏感模块使用 C++ / WebAssembly；美术生产使用 Houdini、Blender；空间查询/碰撞类能力中出现 three-mesh-bvh。

性能上，它依靠一套组合拳：小球形世界、最多 10 人房间、无文字聊天、统一 16x16 color atlas、优化植被几何、自定义 LOD、资产拆分/压缩、WebGL 内 UI、Wasm glyph 文本生成，以及针对 iPhone Safari 的主动内存回收。

一句话定性

Messenger 的工程价值在于：用受控设计和强风格化美术把 Web 端 3D 游戏的复杂度压低，再用 Three.js/WebGL 自研管线精细控制加载、内存、渲染和联机开销。

2. 这是什么游戏

维度	调研结论
类型	浏览器内运行的 3D delivery / exploration / social-lite 小游戏。
核心体验	玩家扮演 messenger / courier，在小型球形星球上完成配送任务，并通过任务路线探索不同区域。
空间设计	世界是一个小星球：城市、住宅区、山地、海边等区域被压缩在同一颗可绕行星球上。
多人方式	在线房间最多约 10 名玩家；互动方式以 3D emoji / stamps 为主，没有传统文字聊天。
社交氛围	更偏 calm / weird / gentle 的共享空间，而不是任务密集型 MMO 或竞技游戏。
商业/分发	免费，主要通过浏览器访问；媒体资料称 PC 和手机浏览器均可游玩。

从设计角度看，Messenger 并不追求大型开放世界，也不把多人同步做到复杂 MMO 规模。它把体验聚焦在短流程配送、视觉探索和低摩擦社交上，这直接降低了运行时、网络、UI 和渲染负担。

3. 技术栈概览

层级	调研结论	证据强度
运行平台	浏览器；PC 和移动端可玩；公开资料称其为 WebGL / browser game。	官方/媒体
3D 渲染库	Three.js + WebGL。Awwwards case study 与 Communication Arts 资料均指向 Three.js。	官方/媒体
游戏引擎	未看到 Unity/Godot build 证据；更像自研 lightweight game framework。社区技术拆解也称其没有使用 Unity/Godot。	官方资料 + 社区观察
语言/模块	vanilla JavaScript、C++、WebAssembly。文字 glyph 生成等局部性能敏感任务使用 Wasm。	官方
空间查询/碰撞	公开 stack 出现 three-mesh-bvh；该库通常用于加速 Three.js mesh raycast、空间查询、碰撞/拾取等。	官方 + 技术推断
美术工具	Houdini、Blender；UI / 视觉设计中出现 Figma、Photoshop。	官方
联机后端	官方确认 backend/networking 基本自研；WebSocket / Node.js 信息来自社区拆解，需按二级来源看待。	官方 + 二级来源

4. 运行时判断

更准确地说，Messenger 不是某个商业游戏引擎运行时，而是：

```
浏览器 JavaScript Runtime
+ ES module 前端 bundle
+ Three.js / WebGL 渲染层
+ Abeto 自研游戏系统
+ 局部 C++ -> WebAssembly 模块
```

公开资料列出的 code stack 包括 vanilla JavaScript、C++、Three.js；性能说明里提到 UI 直接画在 WebGL 里，文字 glyph 由 WebAssembly 生成再交给 GPU 渲染。这说明它至少在某些性能敏感模块上使用了 Wasm，但主体运行环境仍是浏览器。

页面形态上，曾可观察到现代前端 bundle 特征，例如 type="module" 脚本和 runtime chunk；但仅凭这些文件名不能严谨断定具体 bundler 是 Vite、Rollup、Webpack 或其他工具。

5. 渲染管线判断

公开资料没有给出完整 render graph，因此不能精确复原 pass 顺序、shadow strategy、post-processing composer 或 MRT/G-buffer 细节。比较稳妥的判断是：它使用 Three.js WebGLRenderer 体系，配合 Abeto 自定义 shader、outline、水体、植被和 WebGL UI。

```
Houdini / Blender 建模
-> 拆分、压缩、LOD 资源
-> Three.js scene graph
-> 自定义 camera / controls / spherical-world logic
-> 自定义材质与 shader
    - stylized / toon 色块
    - custom outline
    - water shore ripples / gradients
    - vegetation shader
-> WebGL 内 UI + Wasm glyph 文本
-> 浏览器 canvas 输出
```

我倾向判断它是 forward / material-driven 的 WebGL 管线，而不是复杂 deferred rendering。原因是 Three.js 默认 WebGLRenderer 与移动端友好的 stylized browser game 更匹配；项目也强调低资产成本、统一色板和风格化 shader，而没有公开出现 WebGPU、Unity URP/HDRP 或 deferred/G-buffer 证据。这一点是技术判断，不是官方明确披露。

6. 它如何节省性能

6.1 从游戏设计上控制预算

小球形星球是最重要的性能设计之一。它让世界可绕行，但可见范围和内容规模天然受限，减少传统开放世界常见的远景、边界墙、无限地形和大规模 streaming 问题。

多人房间限制在约 10 人，且不提供传统文字聊天，只通过 3D emoji / stamps 交流。这减少了 avatar 同屏复杂度、网络同步压力、聊天 UI 复杂度和内容审核成本。

6.2 球形世界生产流程：unwrapped cube

官方资料提到他们早期尝试程序化 deformation，但最后采用 unwrapped cube 工作流：先像编辑展开立方体一样制作世界，再映射成球。这让道路、建筑、地形和物件摆放更可控，也更利于分区、LOD、碰撞简化和资源压缩。

6.3 16x16 color atlas 降低贴图成本

官方明确提到，世界颜色来自一个 16x16 color atlas texture。这意味着它不是给每个物体准备大张 albedo 贴图，而是用小色板统一控制颜色。好处是显存占用低、加载快、风格统一，也更适合移动端浏览器。

6.4 植被：从 blob 到优化几何 + shader

植被没有走高密度草叶/树叶资产堆叠，而是使用 smart vegetation blobs 来放置，再替换成优化过的 foliage / grass geometry，并用 shader 维持风格。这能显著降低 draw call、顶点量和材质复杂度。

6.5 自定义 LOD

官方性能章节提到 custom LOD system，目标是在远距离简化时仍保持 silhouette 和 volume，降低 popping。Awwwwards 页面图注还提到 4 levels of detail。LOD 策略使移动端和旧设备能够以更低顶点/像素成本渲染远处内容。

6.6 换装与同步的渐进式更新

多人 avatar 换装会触发 mesh/material/model update。官方说他们没有一次性同步全部更新，而是把 model updates “drip in”，也就是分批、渐进式应用，以避免瞬间卡顿。

6.7 UI 直接在 WebGL 中渲染

官方说明 UI 直接绘制在 WebGL 中；文字 glyph 由 WebAssembly 生成，再由 GPU 渲染。这减少了 DOM 布局、CSS 重排和多层合成开销，也让 UI 与 3D 世界风格更一致。

6.8 针对 iPhone Safari 的内存生命周期管理

Abeto 特别提到 iPhone Safari 的内存限制很严，因此他们追踪每个 asset，并在不需要时立即清理 unused assets。同时还写了自定义工具来拆分和压缩模型、贴图。

6.9 压缩格式与社区观察

社区抓包观察提到初始传输量约 5.7 MB、总量约 17.5 MB；也有人从 network tab 看到 .ktx2、.drc、.wasm、Three.js、Web Worker 等迹象，并推测模型使用 Draco 压缩、纹理使用 KTX2 / Basis 类 GPU 压缩格式。由于这些来自社区观察，不应当等同于官方白皮书，但与官方 “split and compress models and textures” 的说法一致。

7. 可借鉴的工程原则

原则	在 Messenger 中的体现	对类似项目的启发
先用设计省性能	小星球、短流程、10 人房间、无文字聊天。	先限制世界规模、社交规模和可见复杂度，再做工程优化。
用风格化代替重资产	16x16 color atlas、toon / stylized shader、custom outline。	不要为了“好看”默认上 PBR、大贴图 and 重后处理。
资产生命周期可观测	追踪每个 asset，不用时立即释放。	Web 端尤其是 iOS Safari 中，内存回收策略必须显式设计。
LOD 不只是减面	保持 silhouette 和 volume，降低 popping。	移动端 3D 项目应把 LOD 当成美术/技术共同生产流程。
UI 与渲染统一	UI 在 WebGL 内绘制，Wasm 生成 glyph。	复杂实时 3D UI 可以减少 DOM 层依赖，避免布局/合成成本。
联机功能少而准	只保留表情/stamps 与少量玩家同步。	多人体验不一定需要完整聊天、语音或 MMO 架构。

8. 证据边界与不确定项

- Three.js 具体版本：未查到可靠公开信息。
- 精确 bundler：能观察到现代前端 bundle / runtime chunk 特征，但不能严谨推出 Vite、Webpack、Rollup 或其他具体工具。
- 完整 render graph：没有官方公布 pass 顺序、shadow strategy、post-processing composer、MRT/G-buffer 等细节。
- Node.js / WebSocket：WebSocket / Node.js 来自社区技术拆解；官方只确认 custom networking code / backend。
- Draco / KTX2 / Basis：来自社区抓包与 network tab 观察；可信度中等，但不是官方正式架构文档。

最稳妥表述

运行时是浏览器 + 自研 JS/Wasm 游戏层；渲染是 Three.js/WebGL + 自定义 shader/outline/UI 管线；性能核心来自美术约束、LOD、压缩、资产生命周期管理和小规模多人设计。

9. 参考来源

[1] Awwwards - Messenger case study: <https://www.awwwards.com/messenger.html>

项目玩法、code stack、性能优化、16x16 color atlas、LOD、WebGL UI、Wasm glyph 等。

[2] Communication Arts - Messenger webpick: <https://www.commarts.com/webpicks/messenger>

Abeto 访谈、Three.js、custom backend/networking、小星球设计等。

[3] AUTOMATON - Messenger news

article: <https://automaton-media.com/articles/newsjp/messenger-20250927-359472/>

玩法、delivery jobs、浏览器/移动端可玩等媒体信息。

[4] WebGPU/WebGL Community showcase - Messenger: <https://www.webgpu.com/showcase/messenger/>

社区技术拆解：Three.js/WebGL、自研方案、WebSocket/Node.js 等二级来源。

[5] Three.js WebGLRenderer documentation: <https://threejs.org/docs/pages/WebGLRenderer.html>

Three.js WebGLRenderer 的基础说明。

[6] Hacker News discussion: <https://news.ycombinator.com/item?id=45396441>

社区抓包观察：传输体积、.ktx2、.drc、.wasm、Web Worker 等迹象。